



Session 13

By: Prof. Raksha Sharma

Module 2, Week 6
June 6th 2026

Module 2: The Data Handling Toolkit: NumPy and Pandas



Data Aggregation and Grouping **(Diving deeper in data analysis using Pandas)**

Aggregation



- Combining multiple data values to produce a summary.
- Aggregation converts large datasets into meaningful insights.

Functions to perform Aggregation: Sum, Average (Mean), Count, Minimum, Maximum

Raw data Vs Summarized (Aggregated) Data

- Compact
 - Easy to interpret
 - Reveals trends quickly
 - Supports decision making
-
- Similar in functioning as Excel Pivot Table, Pandas give flexibility of associating programming

Aggregation and Grouping



- Often we are interested seeing summary for each sub group in our data.

Solution: Perform aggregation group-wise

- Aggregation and grouping go together

Eg., How many employees are there in each department?

Eg., What is the average salary for each gender in each department?

Real Life Applications of Aggregation



Sales- What is the total sales per city from a sales record?

Employee- What is the average salary for each department from employee data?

Education- What are the average, minimum and maximum marks scored in a course?

Healthcare - Which hospital treated the most patients?

Ultimately helps to answer high-level questions:

Which are the Highest and lowest performing groups?

First thing to do



1. Import Pandas -> create/take data in a data-frame
 - You have already created dataset
2. Get to know the functions of Pandas allowing Aggregation or generation of summary

df.groupby(...)

Example Code: `df.groupby("Department")["Salary"].mean()` - Will generate average of the salary for each department

https://colab.research.google.com/drive/1vpYuOwlZlOYEkmPDKOX6_mHoUReiuInO?usp=sharing

Aggregating functions



Use `groupby()` to organize data into meaningful groups.

Apply aggregation functions such as `sum()`, `mean()`, `count()`, `min()`, and `max()`.

Perform multiple aggregations simultaneously using `agg()`

Function	Purpose
<code>sum()</code>	Total value
<code>mean()</code>	Average value
<code>count()</code>	Number of records
<code>min()</code>	Smallest value
<code>max()</code>	Largest value

Aggregating by multiple columns



Datasets often require grouping by more than one category.

- What is average salary for each gender in each department?

```
df.groupby(["Department", "Gender"])["Salary"].mean()
```

Department	Gender	Average Salary
HR	Female	60000
HR	Male	50000
IT	Female	90000
IT	Male	80000

Reset and rename



Index: After grouping, group names become the index to present the group results in python, 'reset' it to new names using `reset_index()`.

- `department_summary = df.groupby("Department")["Salary"].mean().reset_index()`

Column names: Initially it borrows column names from parent table for the aggregation.

- `department_summary = department_summary.rename(columns={"Salary": "Average_Salary"})`



Thank you!